

Capstone Project

Recommending Products to E-commerce Shoppers

A two-stage recommender on H&M transaction data: problem framing, EDA, models, explainability, bias audit, deployment and GenAI

Matthew Martinez

Pillar 5 Capstone, eCommerce domain: recommend products to users

October 2026

GitHub: github.com/mix-afk/Matthew_Martinez_Capstone

Summary

I built a recommender that picks 12 products for each customer for the coming week, using H&M's public transaction data. It works in two stages: seven simple recommenders each suggest items, then a LightGBM model ranks all suggestions by how likely the customer is to buy them.

On a test week the model never saw (16 to 22 Sep 2020, 14,023 customers), the final model scored MAP@12 of 0.0309, which is 3.3 times the best-seller list (0.0094). It finds at least one real purchase for 13.1% of customers vs 7.1% with best sellers.

SHAP shows the model leans on last week's sales, other colors of products the customer already bought, and how recently both the item and the customer were active. The bias audit found that quality goes up with age (MAP@12 0.0283 for 16 to 24 vs 0.0382 for 55+) and that 95.3% of recommendation slots go to the top 20% of products. Removing the age features narrows the age gap at almost no cost, so that is the version I would ship.

Everything is in a public GitHub repo: reproducible code, five notebooks, saved models, MLflow tracking, a FastAPI app with a demo page, Docker, CI tests, and an LLM layer that writes a one-line reason for each recommendation.

Step 1: Problem Understanding and Framing

Frame the business and data science problem clearly. Define whether it's a classification, regression, recommendation, anomaly detection, or clustering task. Specify success metrics (e.g., Accuracy, AUC, RMSE, Silhouette Score) and business KPIs (e.g., cost savings, uplift). Deliverable: Clear problem statement + task type + target metric.

Business problem

An online fashion store has more than 100,000 products, but a shopper only looks at a few dozen per visit. Most stores fill the "You may also like" row, the app home page and promo emails with the same best sellers for everyone. Those slots get wasted: the shopper has already seen the best sellers, and smaller or newer products never get shown.

At Dynamix, an end-to-end e-commerce enabler based in Quezon City, Philippines that helps brands grow and convert on Shopee, Lazada, and TikTok Shop, we make the same kind of call every week: which items to bundle, which add-on deals to push, and which items to flex in a live selling session. Our client data can't be made public, so for this project I used H&M's public data, which has the same shape: customers, products and orders over time.

Problem statement

Given each customer's purchases in the last 8 weeks, product details and basic profile data, rank products so that each customer's top 12 contains as many as possible of the items they actually buy in the next 7 days.

Task type

This is a recommendation task. I solved it as learning to rank in two stages:

1. Candidate generation: seven simple recommenders each propose items, about 80 per customer in total. This cuts 105,000 products down to a list small enough to score
2. Ranking: a binary classifier predicts "will this customer buy this item next week" for every candidate. Sorting by that probability gives the top 12

So the model inside is a classifier, but the output and the main metric are about ranking. I also tested a pure ranking objective (LambdaRank) to check that choice.

Success metrics

1. MAP@12 (primary): mean average precision over the top 12, the official metric of the H&M Kaggle competition. It rewards putting the right items near the top
2. Recall@12 and NDCG@12: how many of the purchased items we found, and how high they were ranked
3. Hit rate: share of customers with at least one purchased item in their 12
4. Catalog coverage: share of products that get recommended to at least one customer
5. AUC: how well the ranker separates bought vs not bought candidates
6. Candidate recall: share of next-week purchases that make it into the candidate pool at all. This is the ceiling for stage 2

Business KPIs

Offline metrics tell me if the model is better at predicting purchases. In a live test I would track:

1. Click-through and add-to-cart rate on recommendation slots
2. Conversion and revenue per session from recommended items, as uplift vs a best-seller list in an A/B test
3. Items per order, from cross-sell (other colors, items bought together)
4. Long-tail exposure: share of slots that go to products outside the top 20% sellers, which matters for smaller brands
5. Fairness: hit rate of every age band at 80% or more of the best band

Target

Beat the best-seller baseline on MAP@12 by a clear margin on a week the model never saw, with no age band below 0.8 disparate impact.

Step 2: Data Collection and Understanding

Use public datasets (Kaggle, UCI, APIs, etc.) or approved custom data. Summarise feature types, missing values, outliers, etc. Provide a data dictionary (variables, types, units, allowed values). Deliverable: Dataset overview + data dictionary.

Dataset and why I picked it

Source: H&M Personalized Fashion Recommendations, a 2022 Kaggle competition with real H&M data. I used a parquet copy of the same files (Kaggle dataset by Derrick Mwiti) because it is smaller to download. I checked the row counts against the official competition and they match: 31,788,324 transactions, 1,371,980 customers and 105,542 articles.

1. Real retailer data over two years, not a toy dataset

2. Many repeat customers, so collaborative filtering has something to learn from. In many marketplace datasets most buyers order only once
3. Customer age is included, which I need for the bias audit
4. Rich product attributes (type, group, color, pattern) for content-based features

Table 1. The three source tables (full data)

Table	Rows	Columns	What it holds
transactions_train	31,788,324	5	One row per unit sold: date, customer, article, price, sales channel
customers	1,371,980	7	Customer id, newsletter flags, club status, age, hashed postal code
articles	105,542	25	Product variant id, parent product, type, group, color, pattern, department, description

Full data overview

- 20 Sep 2018 to 22 Sep 2020, two full years
- Sales channel 2 (generally read as online) is 70.4% of transactions
- Store sales (channel 1) drop to zero from about late March to May 2020, when stores closed for COVID
- Big spikes are sale days, and the late June 2020 sale is the largest in the modeling window

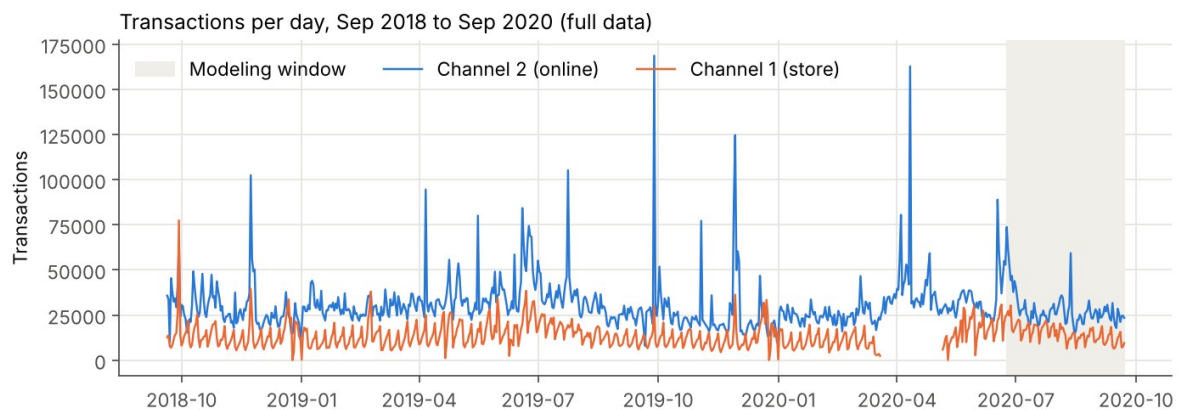


Figure 1. Transactions per day over the full two years. The gray band is the 13-week modeling window.

Sample

Two years and 31.8 million rows is too much for my 8 GB workspace, and it is not needed: fashion moves fast, and what someone bought in 2018 says little about September 2020. So I kept:

1. The last 13 weeks: 24 Jun to 22 Sep 2020
2. About 20% of customers, picked by a fixed rule. Customer ids are hex hashes, so I kept ids ending in 00 to 33 (52 of 256 values). It acts like a random sample but gives the same customers on every run, without a seed

Result: 789,049 transactions, 278,736 customers (20.3% of all), 106,650 of whom bought something in the window, and 31,178 products sold. Each week of the window has a role:

Table 2. Time-based split of the sample

Week	Dates (2020)	Units	Buyers	Role
0	16 to 22 Sep	49,001	14,023	Test (used once, at the end)
1	9 to 15 Sep	52,332	14,753	Validation (tuning and model choice)
2 to 4	19 Aug to 8 Sep	163,884	46,354 (customer-weeks)	Training labels
5 to 12	24 Jun to 18 Aug	523,832		History only (features)

Feature types

- Identifiers: customer_id (64-character hash), article_id and product_code (integers)
- Dates: t_dat (purchase day)
- Numeric: price (scaled by H&M, not real money), age (years)
- Binary flags: FN (gets fashion news), Active, sales channel
- Categorical: club status, newsletter frequency, and 11 product attributes (type, group, pattern, color, color tone, color family, department, index, index group, section, garment group)
- Free text: product name and description (only used for display)

Missing values

Table 3. Missing values in the sample (transactions have none)

Table	Column	Missing	Missing %	How I handled it
customers	FN	181,680	65.18%	Missing means not subscribed, set to 0
customers	Active	184,290	66.12%	Missing means not active, set to 0
customers	club_member_status	1,218	0.44%	Set to UNKNOWN
customers	fashion_news_frequency	3,275	1.17%	Merged with NONE
customers	age	3,241	1.16%	Kept for the audit (band Unknown), median 32 for the model, plus a missing flag
articles	detail_desc	416	0.39%	Empty text (not used by the model)

Duplicates

- 66,416 transaction rows (8.4%) are exact duplicates. These are not errors. H&M writes one row per unit, so two identical rows mean two units of the same item on the same day. 57,332 order lines have more than one unit, the most is 23
- No duplicate customer ids or article ids

Outliers

- Price: median 0.0244, 99th percentile 0.0847, max 0.5068. The IQR rule flags 30,951 rows (3.9%). These are real expensive items like coats, so I kept them. Tree models are not hurt by them, and the model mostly uses price as a ratio to the customer's usual price

- Age: 16 to 99. The IQR rule flags 42 customers aged 87 or more. Rare but possible, so kept

Distributions



Figure 2. Customer age is bimodal (peaks around 21 to 25 and around 50). Prices are right-skewed. Most customers buy only a few units in 13 weeks.

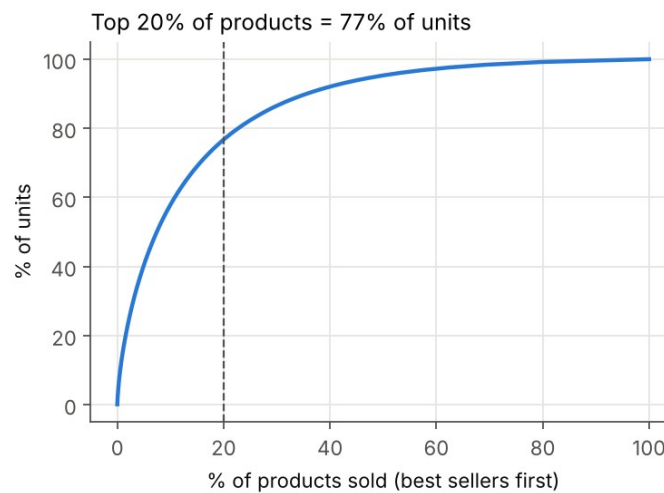


Figure 3. Sales are concentrated: the top 20% of products make up 77% of units.

Data dictionary

The full version is in docs/DATA_DICTIONARY.md in the repo. All ranges and counts below were recomputed on the full files.

Table 4. Data dictionary of the source tables

Table	Variable	Type	Unit	Allowed values / range	Description
transactions	t_dat	date (string in raw file)	day	2018-09-20 to 2020-09-22 (sample: 2020-06-24 to 2020-09-22)	Purchase date. One row per unit sold
transactions	customer_id	string (64-char hex hash)		Matches customers.customer_id	Anonymized customer key
transactions	article_id	int64		Matches articles.article_id	Product variant key (product + color)
transactions	price	float64	scaled currency	0.00002 to 0.5915 (scaled by H&M, not real money)	Unit price paid
transactions	sales_channel_id	int64		1, 2 (2 is widely read as online, 1 as store)	Sales channel

Table	Variable	Type	Unit	Allowed values / range	Description
customers	customer_id	string (64-char hex hash)		Unique per row	Anonymized customer key
customers	FN	float64 (flag)		1.0 or missing (missing = 0)	Customer gets fashion news
customers	Active	float64 (flag)		1.0 or missing (missing = 0)	Customer is active for communication
customers	club_member_statuses	string		ACTIVE, PRE-CREATE, LEFT CLUB, missing	Loyalty club status
customers	fashion_news_frequency	string		NONE, None, Regularly, Monthly, missing (None and missing merged into NONE)	Newsletter frequency
customers	age	float64	years	16 to 99, about 1.2% missing	Customer age. Used for the fairness audit
customers	postal_code	string (hash)		Hashed, 352,899 distinct values	Postal code. Dropped (too granular, possible location proxy)
articles	article_id	int64		105,542 unique	Product variant key
articles	product_code	int64		47,224 unique	Parent product (all colors of one item share it)
articles	prod_name	string		Free text	Product name
articles	product_type_name	string		131 values, e.g. Trousers, Sweater, Dress	Product type (paired code: product_type_no)
articles	product_group_name	string		19 values, e.g. Garment Upper body, Accessories	Product group
articles	graphical_appearance_name	string		30 values, e.g. Solid, Stripe, All over pattern	Print or pattern
articles	colour_group_name	string		50 values, e.g. Black, Light Beige	Detailed color (paired code: colour_group_code)
articles	perceived_colour_value_name	string		8 values, e.g. Dark, Light, Dusty Light	Color tone
articles	perceived_colour_master_name	string		20 values, e.g. Black, Blue, Beige	Main color family
articles	department_name	string		250 values	Buying department
articles	index_name	string		10 values, e.g. Ladieswear, Divided, Menswear	Store index
articles	index_group_name	string		5 values: Ladieswear, Baby/Children, Divided, Menswear, Sport	Top-level index group
articles	section_name	string		56 values	Section within index
articles	garment_group_name	string		21 values, e.g. Jersey Basic, Knitwear	Garment group
articles	detail_desc	string		Free text, 416 missing	Product description

Step 3: Data Preprocessing, EDA and Feature Engineering

Clean data: Handle nulls, duplicates and outliers. Engineer features: Scaling, encoding, binning and domain-derived features. Applied EDA: Distributions, relationships, clustering tendency (if unsupervised). Feature importance & explainability: SHAP, LIME, or model-based importances. Feature selection: At least one approach (filter, wrapper, or embedded). Dimensionality reduction: PCA (and t-SNE/UMAP for visualisation if needed). Deliverable: "EDA + Feature Engineering Report" with reproducible code & justifications.

Cleaning

1. FN and Active: missing set to 0, since the only value present is 1
2. fashion_news_frequency: "None" and "NONE" merged, missing set to NONE
3. club_member_status: missing set to UNKNOWN
4. Age: left missing in the raw column so the audit can report an Unknown band. For the model I added age_filled (median, 32) and an age_missing flag
5. postal_code: dropped. It has 352,899 hashed values, which is too granular to learn from, and it is a location proxy I did not want in the model
6. Duplicate transaction rows: kept, because each one is a unit sold
7. Price outliers: kept (see Step 2)
8. Dates parsed, week index added (week 0 is the last week), sales channel turned into an online flag

Code: src/recsys/data.py. Before and after checks are in notebook 02.

Applied EDA

1. Next week's basket is hard to predict from history. Only about 2.6% of the items a customer buys next week were bought by that same customer in the previous 8 weeks, and about 38% of next week's buyers have no purchase in those 8 weeks at all. This sets expectations for the scores later: even a good model will have low absolute MAP on this data.

Table 5. How much of each week's purchases can be traced to the 8 weeks before

Target week	Customer-item pairs	Bought before by same customer	Other color bought before	In last week's top 36	Buyers with 8-week history
0	43,597	2.6%	4.5%	5.5%	61.7%
1	46,881	2.6%	4.4%	4.4%	61.8%
2	47,994	2.5%	4.2%	4.9%	60.7%
3	52,521	2.0%	3.7%	4.9%	61.3%
4	46,688	2.4%	4.6%	4.0%	65.1%

2. Age changes what people buy. Divided, H&M's younger line, is 27% of units for 16 to 24 but 17% for 55+. Ladieswear goes the other way (61% to 72%), and Baby/Children peaks for 35 to 44, which is likely parents. This is why I added an age-band best-seller generator, and also why age needs a fairness check.

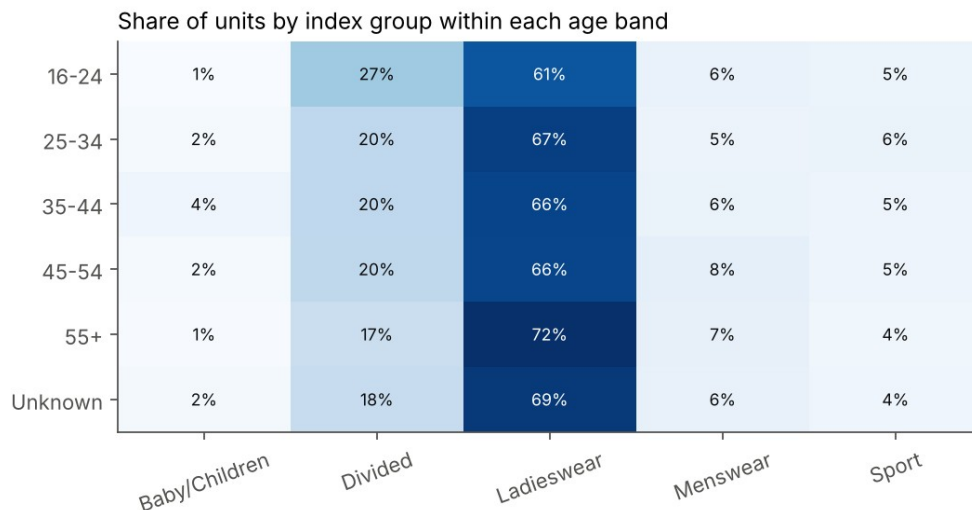


Figure 4. Share of units by index group within each age band.

3. Young customers buy more new things. Of the items bought by 16 to 24 year olds in weeks 0 to 2, 2.0% were items they had bought before and 3.6% were another color of something they owned. For 55+ those numbers are 2.8% and 5.0%. This becomes important in the bias audit.

4. Volume is steady at about 49,000 to 67,000 units a week in the sample, except the late-June sale week (105,000). The online share moves around 65%.

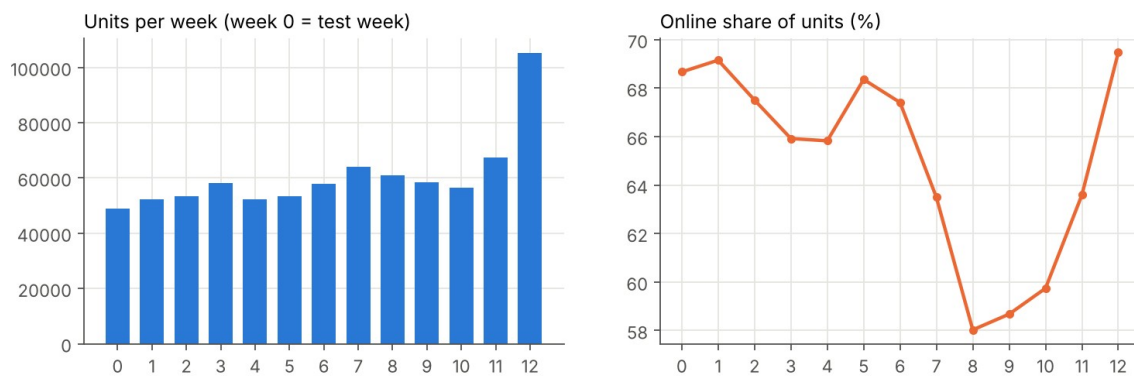


Figure 5. Weekly units and online share in the sample.

Feature engineering

For every customer and candidate item I built 50 features, all from the 8 weeks before the target week. That rule is what keeps the features free of leakage.

Table 6. The 50 engineered features by group (definitions in reports/tables/feature_list.csv)

Group	Count	Features
Customer x item	10	rep_count, rep_days_since, rep_rank, ci_prodcod_count, ci_ptype_share, ci_igroup_share, ci_garment_share, ci_colour_share, ci_price_ratio, ci_age_gap
Candidate score	11	itemcf_score, itemcf_rank, als_score, als_rank, content_score, content_rank, sib_pop, sib_rank, pop_rank, bandpop_rank, n_sources
Item	11	a_pop_1w, a_pop_2w, a_pop_4w, a_pop_8w, a_trend, a_buyers_4w, a_price_mean, a_online_share, a_buyer_age_mean, a_days_since_first, a_days_since_last

Group	Count	Features
Customer	13	c_n_tx, c_n_tx_2w, c_n_articles, c_n_weeks, c_days_since_last, c_price_mean, c_online_share, c_age, c_age_missing, c_fn, c_active, c_club_active, c_news_regular
Item (categorical)	5	product_group_name, index_group_name, garment_group_name, perceived_colour_master_name, graphical_appearance_name

Some features come straight from how people shop for clothes:

1. Other colors: shoppers who like a cut often buy it again in another color. ci_prodcode_count counts past purchases of the same parent product, and a generator proposes the other variants
2. Trend: a_trend is last week's sales divided by the 4-week weekly average, to catch items that are taking off
3. Budget fit: ci_price_ratio is the item's average price divided by the customer's usual price
4. Taste shares: share of the customer's past purchases in the item's product type, index group, garment group and color family
5. Age fit: ci_age_gap is the gap between the customer's age and the average age of the item's buyers

Scaling, encoding and binning:

- Logistic regression: median imputation with missing-value indicators, StandardScaler, one-hot encoding of the 5 product categoricals (rare levels under 200 rows grouped)
- LightGBM: no scaling needed; categoricals passed as native categorical features; missing values handled by the trees
- Binning: age into 16 to 24, 25 to 34, 35 to 44, 45 to 54, 55+ and Unknown, for the age-band generator and the audit

Class imbalance: 1.40% of training rows are purchases (about 70 negatives per positive) and 0.43% in the test week. I deal with this in Step 4 by using ranking metrics instead of accuracy.

Correlations: a few features are near duplicates: FN vs newsletter regular (0.997), 4-week sales vs 4-week buyers (0.993), FN vs Active (0.986), units vs distinct items bought (0.968).

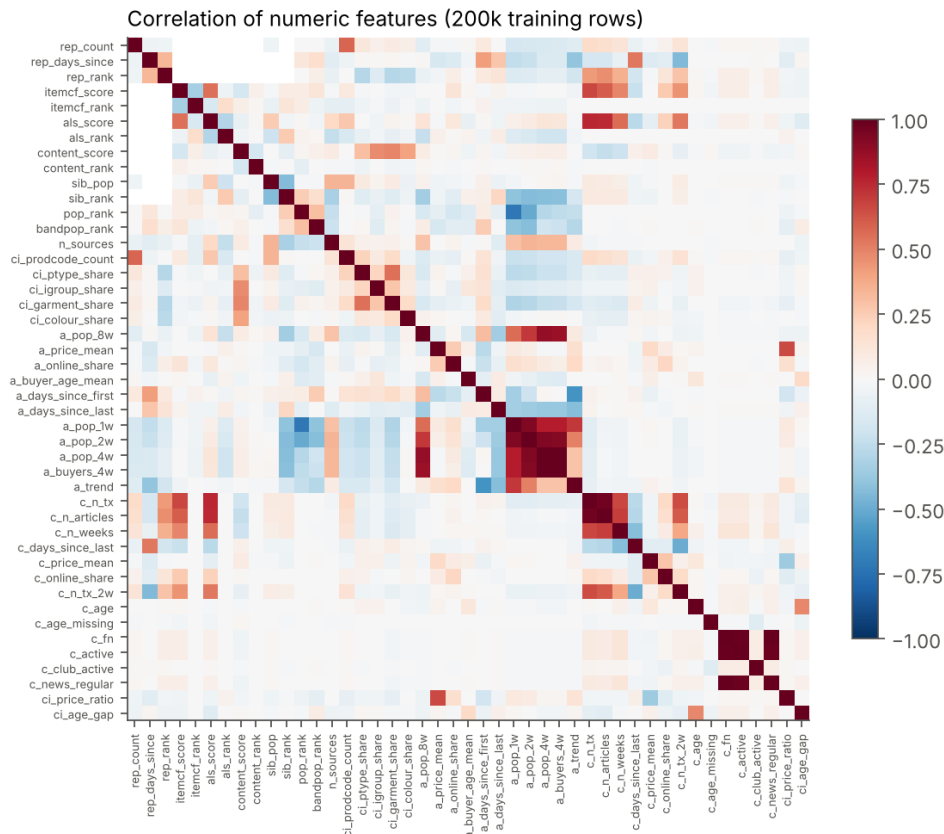


Figure 6. Correlation of the numeric features on 200,000 training rows.

Feature selection

I tried a filter method and an embedded method on a 200,000-row training sample, then checked each selected set with the same LightGBM settings on the validation week.

1. Filter: a variance threshold removed nothing. Mutual information (with low-cardinality columns treated as discrete) ranks the generator scores highest: `als_score`, `content_score`, `itemcf_score`, `rep_rank`, `sib_rank`, `bandpop_rank`. 12 of the 13 customer-only features rank in the bottom half, because their values are the same for all of one customer's candidates, so on their own they cannot tell items apart
2. Embedded: L1-regularized logistic regression ($C = 0.01$) kept 22 of the 45 numeric features. The strongest were days since the item last sold (negative), days since the customer bought the item (negative), trend (positive), customer's recent volume and repurchase count

Table 7. Selected feature sets vs all features (same LightGBM settings, validation week)

Feature set	Features	Validation MAP@12	Validation AUC
all features	50	0.0301	0.7656
top 25 by mutual information + categoricals	30	0.0303	0.7640
L1 kept + categoricals	27	0.0291	0.7566

Decision: the top 25 by mutual information matched the full set (0.0303 vs 0.0301, a difference well inside week-to-week noise), while the L1 set lost about 3%. I kept all 50 features in the final model for two reasons. First, the filter looks at one feature at a time and drops 12 of the 13 customer features, yet SHAP (Step 5) shows customer recency is the 6th

most useful feature once trees can combine it with item features. Second, the fairness tests in Step 5 need the age features inside the model so I can measure what happens when they are removed. If serving speed mattered more, the 30-feature set is a safe swap.

Dimensionality reduction

PCA on product attributes: the 11 categorical product columns one-hot encode into 600 columns. I compressed them with PCA to 24 components, which keep 60% of the variance (reaching 90% would take more than 80 components, because the attributes are spread out). The 24-number vectors make the content-based generator fast: a customer's taste is the recency-weighted average of the vectors of what they bought, and items are scored by cosine similarity. The first two components already separate Ladieswear from Divided and Menswear.

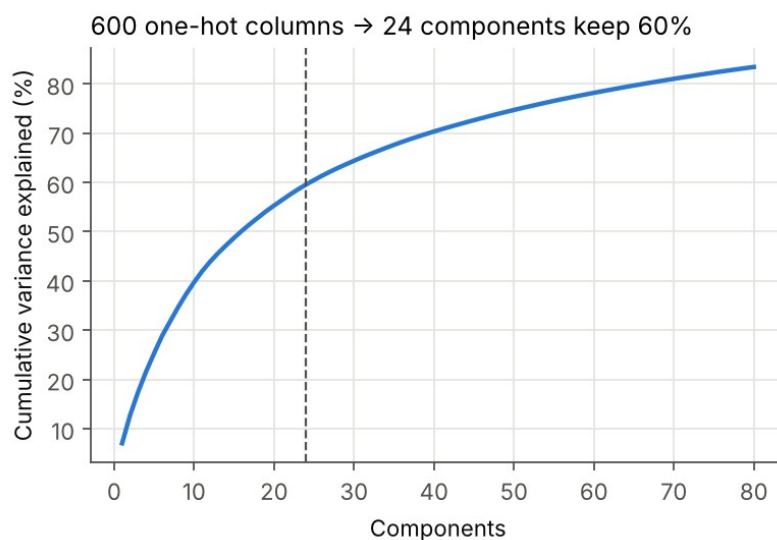


Figure 7. Cumulative variance explained by PCA on the 600 one-hot product columns.

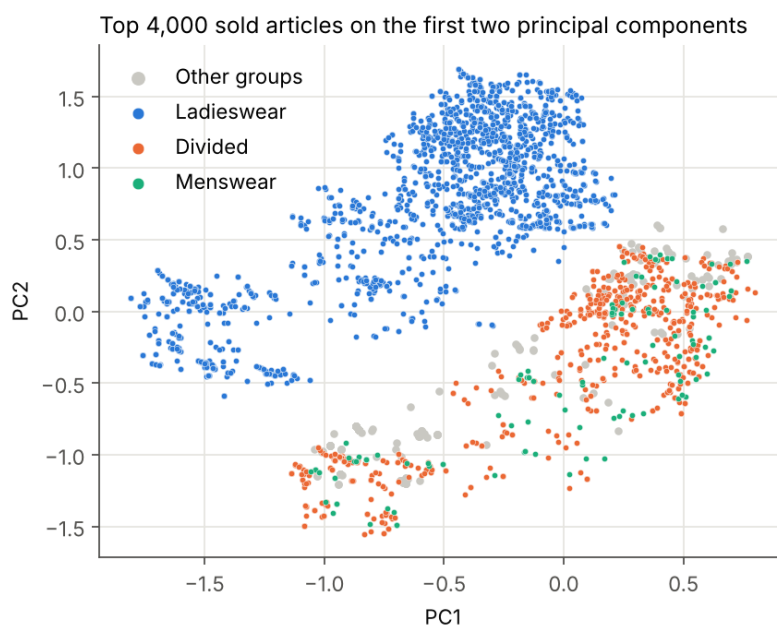


Figure 8. The 4,000 best-selling articles on the first two principal components.

t-SNE on ALS item vectors: ALS learns item vectors from purchases only, without seeing any product attributes. Projected with t-SNE, the items still group by index group (Menswear and the Baby/Children and Sport groups form their own islands). So the purchase data carries real style signal, which supports using collaborative filtering.

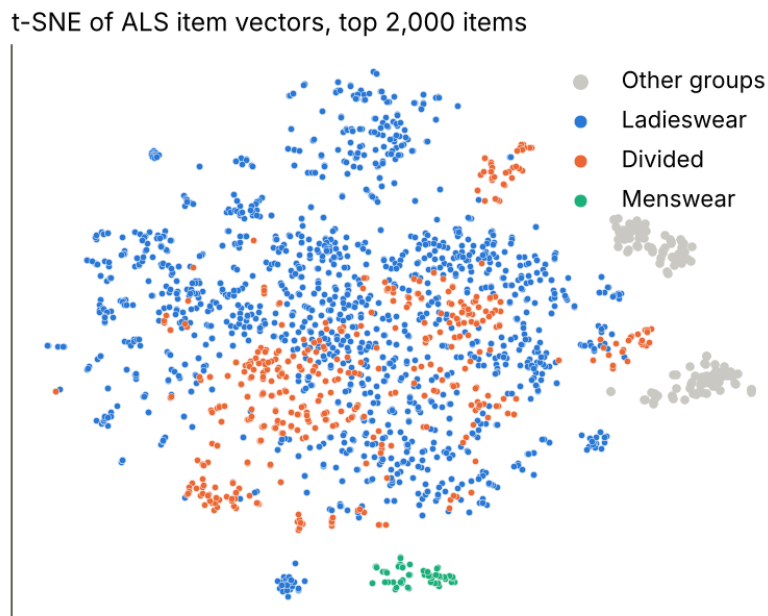


Figure 9. t-SNE of ALS item vectors for the 2,000 best sellers, colored by index group.

Feature importance and explainability are covered with SHAP, PDP and ICE in Step 5, using the final model.

Step 4: Model Implementation and Comparison

Experiment with appropriate models: Supervised: Logistic Regression, Decision Trees, Random Forest, XGBoost, SVM, etc. Unsupervised: K-Means, DBSCAN, Hierarchical (Elbow, Silhouette). Recommendation: Collaborative or content-based. Deep Learning: RNNs, CNNs, LSTMs, Transformers (if appropriate). Evaluation: compare with relevant metrics. Reproducibility: save configs and artefacts (models/). Deliverables: Trained models, metrics and comparison between models.

Setup

- Training labels: weeks 2, 3 and 4. Validation: week 1, used for tuning and choosing the model. Test: week 0, used once at the end
- For every target week, candidates and features come only from the 8 weeks before it
- Training rows: candidates of customers who bought at least one of their candidates, 1,018,522 rows. This gives the model useful contrast. Validation and test keep every customer, so the evaluation matches real use
- Evaluation is over customers who bought something in the target week, the same way the Kaggle competition scored it. Customers with fewer than 12 suggestions are filled with last week's best sellers

Stage 1: seven recommenders (collaborative, content-based and rules)

1. Best sellers: last week's top 36

2. Age-band best sellers: top 12 in the customer's age band over the last 2 weeks
3. Buy again: the customer's own recent items, up to 20
4. Item-to-item collaborative filtering: cosine similarity between items on the customer x item matrix (implicit library, 50 neighbors per item)
5. ALS matrix factorization: 64 factors, 20 iterations, confidence weight 10
6. Content-based: cosine similarity between the customer's PCA taste vector and the 3,000 best sellers of the last 2 weeks
7. Other colors: other variants of products the customer bought, ranked by recent sales

Together they produce about 81 candidates per customer and cover 11.2% of next-week purchases. Best sellers contribute the most coverage (5.5%), then buy again (2.6%) and age-band best sellers (2.2%).

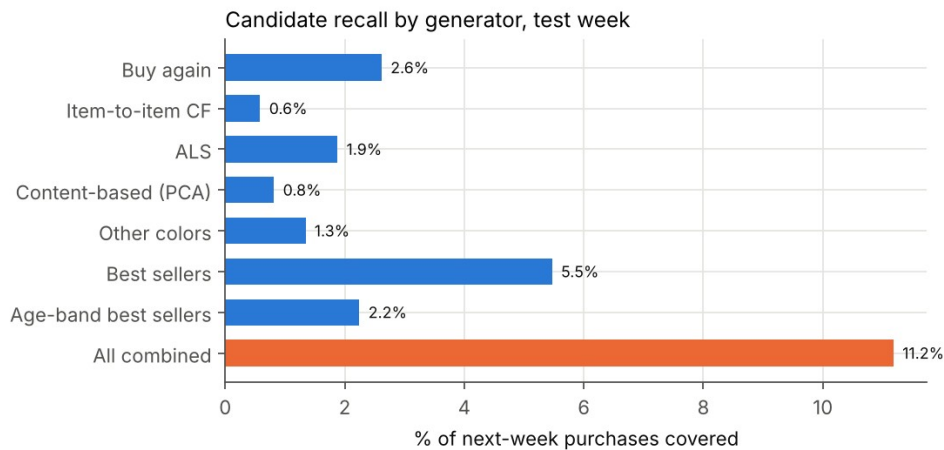


Figure 10. Share of next-week purchases (test week) that each generator puts into the candidate pool.

Table 8. Candidate tables per week

Week	Role	Candidate rows	Customers	Candidates per customer	Positive rate	Candidate recall
0	test	1,142,923	14,023	81.5	0.43%	11.2%
1	valid	1,191,921	14,753	80.8	0.38%	9.8%
2	train	1,224,915	15,368	79.7	0.39%	10.1%
3	train	1,340,176	16,345	82.0	0.37%	9.5%
4	train	1,264,202	14,641	86.3	0.35%	9.5%

Stage 2: three rankers

1. Logistic regression: the baseline. C tuned over 0.1 and 1.0
2. LightGBM classifier: random search over 8 settings (number of leaves, learning rate, minimum rows per leaf, column and row sampling, L2), early stopping on validation AUC
3. LightGBM LambdaRank: a ranking objective that optimizes NDCG directly, 4 random settings, early stopping on validation NDCG@12

I picked the model by validation MAP@12, before looking at the test week.

Table 9. Tuned settings and validation results

Model	Best settings	Validation MAP@12	Validation AUC	Rounds
Logistic regression	C = 0.1	0.0273	0.741	
LightGBM classifier	num_leaves 127, learning_rate 0.05, min_child_samples 200, colsample_bytree 0.7, subsample 0.6, reg_lambda 5.0	0.0305	0.768	137
LightGBM LambdaRank	num_leaves 63, learning_rate 0.05, min_child_samples 50, colsample_bytree 0.9, subsample 0.8, reg_lambda 5.0	0.0301	0.759	132

Test-week comparison

Table 10. All models on the test week (14,023 customers, week 0)

Stage	Model	MAP@12	Recall@12	NDCG@12	Hit rate	Coverage	AUC
Generator	Best sellers	0.0094	0.0284	0.0171	7.1%	0.05%	
Generator	Age-band best sellers	0.0092	0.0271	0.0165	6.6%	0.19%	
Generator	Buy again	0.0263	0.0557	0.0389	11.2%	54.43%	
Generator	Item-to-item CF	0.0049	0.0161	0.0094	3.9%	77.37%	
Generator	ALS matrix factorization	0.0089	0.0271	0.0164	6.5%	7.15%	
Generator	Content-based (PCA)	0.0050	0.0171	0.0097	4.2%	12.36%	
Generator	Other colors	0.0117	0.0338	0.0208	7.8%	30.44%	
Ranker	Logistic regression	0.0293	0.0620	0.0432	12.0%	29.63%	0.730
Ranker	LightGBM classifier	0.0309	0.0663	0.0461	13.1%	29.02%	0.758
Ranker	LightGBM LambdaRank	0.0312	0.0648	0.0457	12.7%	29.32%	0.756

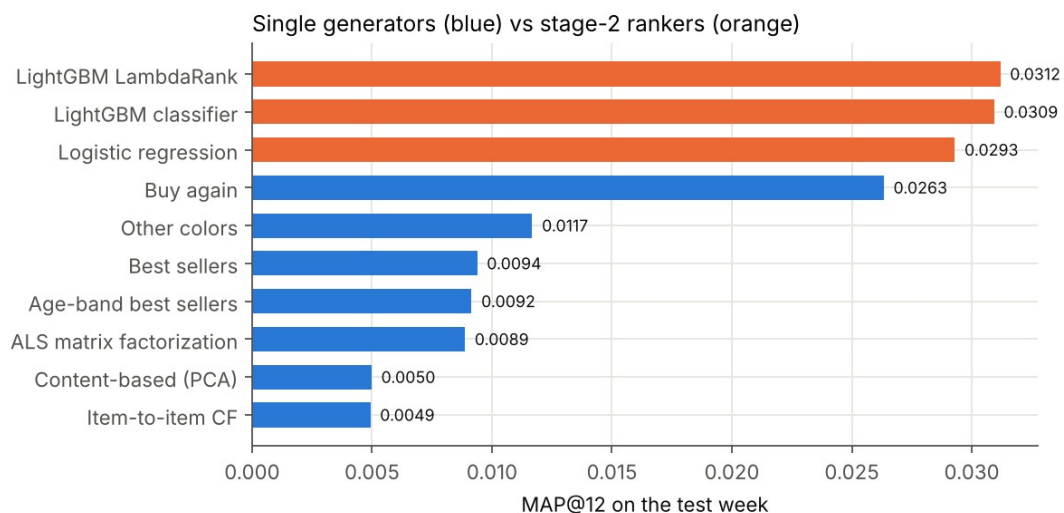


Figure 11. MAP@12 on the test week: single generators vs rankers.

What the results say:

1. Every ranker beats every single generator. The best generator on its own is buy again (0.0263), and the LightGBM ranker adds 17% on top of it

2. The LightGBM classifier is 3.3 times the best-seller list on MAP@12 and nearly doubles the hit rate (13.1% vs 7.1%)
3. LambdaRank ties it on MAP@12 (0.0312 vs 0.0309) but is behind on recall, NDCG, hit rate and AUC. I kept the classifier: it won on validation, and probabilities are easier to explain, set thresholds on and audit
4. Logistic regression gets 95% of the LightGBM score. Most of the signal is in the features; the trees add the interactions
5. For context, a top-2% Kaggle solution (45th of 3,006 teams) reported 0.0300 on the private leaderboard using the full data. It is not directly comparable to my sample, but it shows these numbers are in the normal range for this problem

Warm vs cold customers

Table 11. Test week by whether the customer has any purchase in the 8-week history

Customers	Count	Ranker MAP@12	Ranker hit rate	Best sellers MAP@12
has 8-week history	8,651	0.0448	16.7%	0.0101
no history (cold)	5,372	0.0086	7.4%	0.0083

The ranker earns its keep on customers with history: about 4 times the best-seller score. For customers with no history it is basically the best-seller list, which is expected since it knows nothing about them except age.

Overfitting check

Table 12. Same model scored on the training weeks, the validation week and the test week

Split	Weeks	MAP@12	Recall@12	Hit rate	AUC
train	3	0.0420	0.0778	16.0%	0.856
valid	1	0.0305	0.0642	12.7%	0.768
test	1	0.0309	0.0663	13.1%	0.758

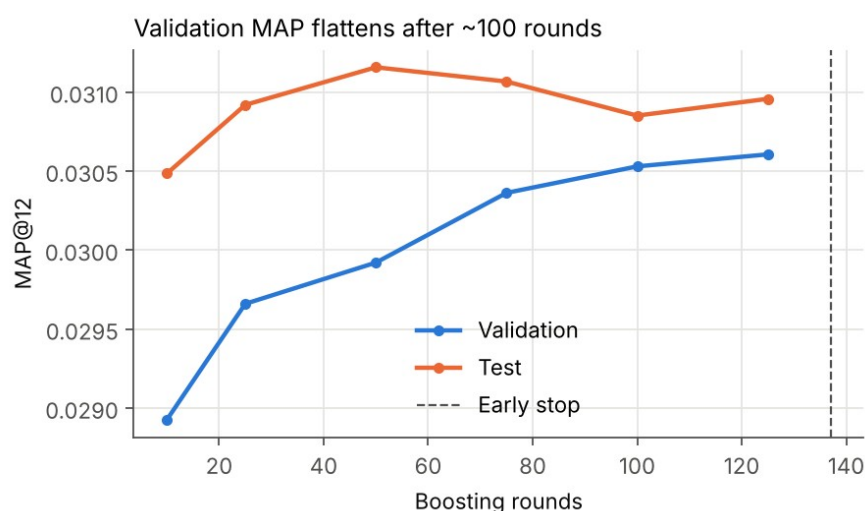


Figure 12. Validation and test MAP@12 by number of boosting rounds.

There is a gap between training (AUC 0.856) and validation (0.768): the trees partly memorize the training weeks. But validation and test agree closely, so the model generalizes

to a new week. Early stopping at 137 rounds, at least 200 rows per leaf and L2 of 5 keep it in check, and validation MAP is flat after about 100 rounds.

Reproducibility

- One config file (configs/config.yaml) holds the dates, sample rule, week split, candidate sizes, tuning budget and fairness settings. Seed 42 everywhere
- models/ holds the three rankers (joblib, plus LightGBM text boosters), model_meta.json (feature list, categories, tuned settings, validation metrics) and test_metrics.json
- Every training run is logged to MLflow (parameters and metrics), and tuning tables are saved in reports/tables/
- The whole pipeline runs with make sample, make tables, make train, make serve

Step 5: Critical Thinking, Ethical AI and Bias Auditing

Explain model decisions (SHAP, LIME, PDP, ICE). Address limitations (imbalance, leakage, overfitting). Bias detection & fairness audits: Check outputs across sensitive groups (gender, race, age, socioeconomic status). Use fairness metrics (demographic parity, equalised odds, disparate impact). Propose mitigations (reweighting, thresholds, augmentation, post-processing). Deliverable: "Bias & Fairness Analysis" section in the final report.

5.1 Explaining the model

SHAP values come from LightGBM's built-in TreeSHAP (exact for tree models) on 6,000 test-week rows: 1,000 bought and 5,000 not bought. I used SHAP instead of LIME because for trees SHAP is exact and gives the same answer every run, while LIME fits a local linear approximation that can change between runs.

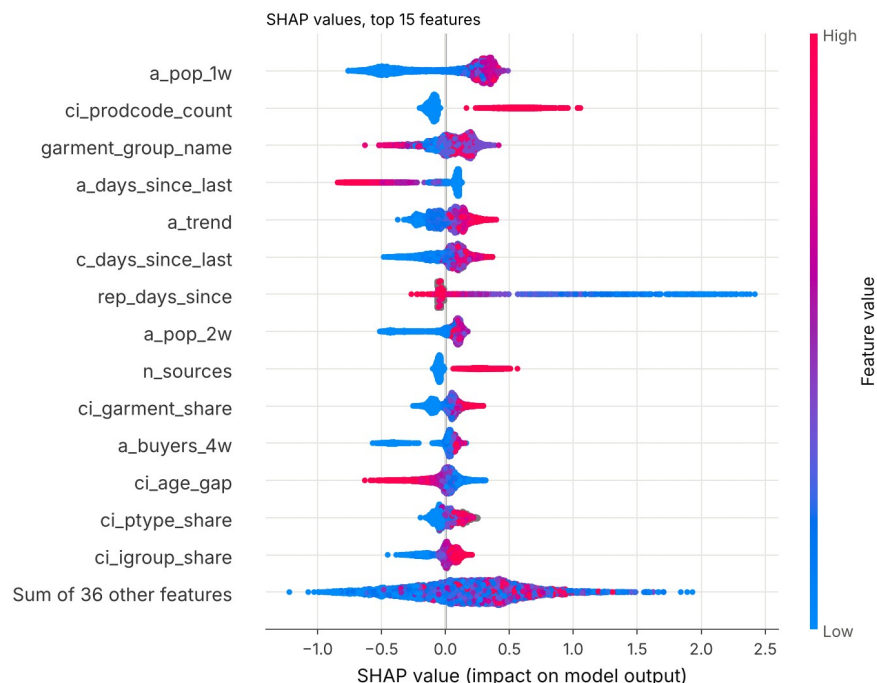


Figure 13. SHAP values for the 15 most important features. Each dot is one customer-item pair; red is a high feature value.

1. Last week's sales (a_pop_1w) is the biggest driver. Items that sold well last week get pushed up, items with low sales get pushed down
2. ci_procode_count: if the customer already bought another variant of the product, the item jumps up
3. Garment group, days since the item last sold (items that stopped selling are pushed down), trend, and how recently the customer was active come next
4. rep_days_since: when a customer bought the exact item recently, the effect is the largest of any feature (up to +2.4), but it only exists for a few rows, so its average is 7th
5. ci_age_gap is 12th: items whose buyers are far from the customer's age get pushed down. This is one way age enters the model, and it is tested in 5.3

LightGBM's gain importance ranks rep_days_since first instead. Gain measures how much a feature is used in splits, while SHAP measures its effect on each prediction, so a rare but strong feature scores high on gain.

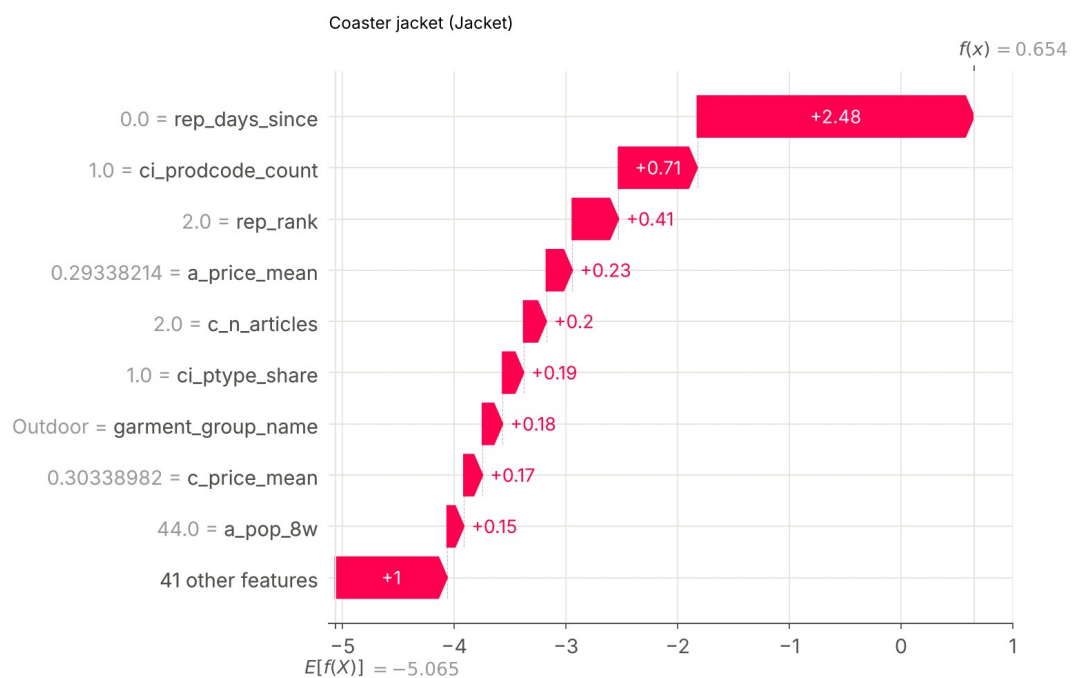


Figure 14. Why one customer's number 1 pick was recommended: a jacket they had bought the day before the target week. They did buy it again.

The waterfall above explains one recommendation. The customer bought the same jacket the day before the target week (rep_days_since = 0), which alone adds 2.48 to the log-odds, and had bought another variant of it too. The predicted chance went from about 1% for an average candidate to about 66%. A repeat buy within days is most likely a size swap or a second unit.

PDP (orange) and ICE (blue) on predicted purchase probability

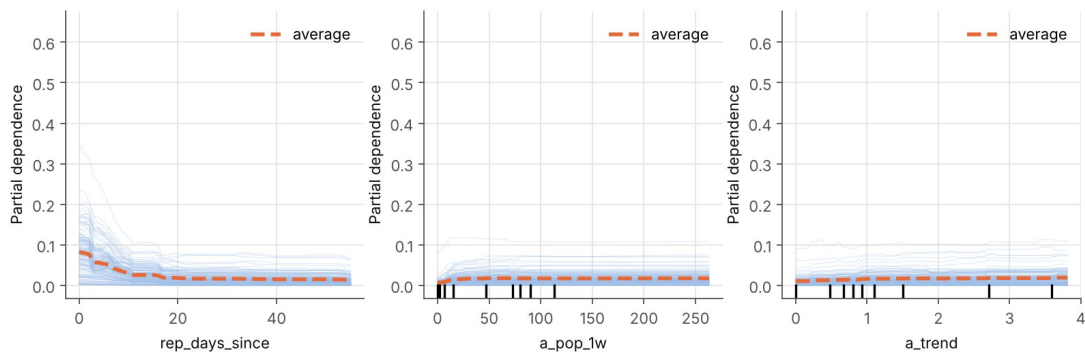


Figure 15. Partial dependence (orange) and individual curves (blue) for three features.

- `rep_days_since`: the purchase chance is highest right after the customer bought the item and falls over the first 2 to 3 weeks. The ICE lines show some customers start from 20% to 35%, so the effect is much stronger for some people
- `a_pop_1w`: the chance rises quickly as an item reaches about 20 units a week in the sample, then flattens. Past that point more sales add nothing
- `a_trend`: items selling faster than their 4-week average get a small, steady lift

5.2 Limitations

1. Class imbalance: only 0.43% of test candidates are bought (233 negatives per positive). Accuracy would be 99.6% by always saying no, so I used ranking metrics and AUC. Training on customers with at least one positive gives the model better contrast
2. Leakage: every feature for a target week only uses the 8 weeks before it, and history always ends the day before the target week starts (checked for all 5 weeks in notebook 04). Product attributes are static. PCA was fit on product attributes only, no labels. The test week was scored once
3. Overfitting: there is a train vs validation gap (Step 4), controlled by early stopping, minimum leaf size and L2. Validation and test agree
4. Candidate ceiling: only 11.2% of next-week purchases are in the candidate pool, so the ranker can never recommend the other 88.8%. Better generators are the biggest lever for improvement
5. Sample: 20% of customers and 13 weeks. Collaborative signals are weaker than on the full data, and absolute scores will shift on the full data
6. Cold start: 38% of test-week buyers have no history in the last 8 weeks and get close to best-seller quality
7. Prices are scaled by H&M and there is no margin data, so I cannot optimize for profit, only for purchases
8. Offline is not online: recommendations change what people see and buy. The real effect needs an A/B test

5.3 Bias and fairness analysis

Which sensitive attributes I could check: the data has age but no gender, race or income. I chose not to guess gender from purchases (for example from Menswear share), because inferring a protected attribute creates its own privacy problem. The postal code could act as a proxy for income, but it is hashed and I dropped it. So the audit covers age on the

customer side and popularity on the item side, which is about fairness to products and the smaller brands behind them.

Customer side: quality by age band. Disparate impact here is each band's hit rate divided by the best band's hit rate. Below 0.8 fails the usual four-fifths rule.

Table 13. Recommendation quality by age band, test week

Age band	Customers	MAP@12	Recall@12	Hit rate	Disparate impact	Below 0.8?
16-24	3,824	0.0283	0.0641	12.0%	0.81	no
25-34	4,242	0.0283	0.0614	13.0%	0.88	no
35-44	1,467	0.0317	0.0626	13.1%	0.88	no
45-54	2,607	0.0335	0.0698	13.8%	0.93	no
55+	1,827	0.0382	0.0801	14.8%	1.00	no
Unknown	56	0.0327	0.0625	10.7%	0.72	too few to judge

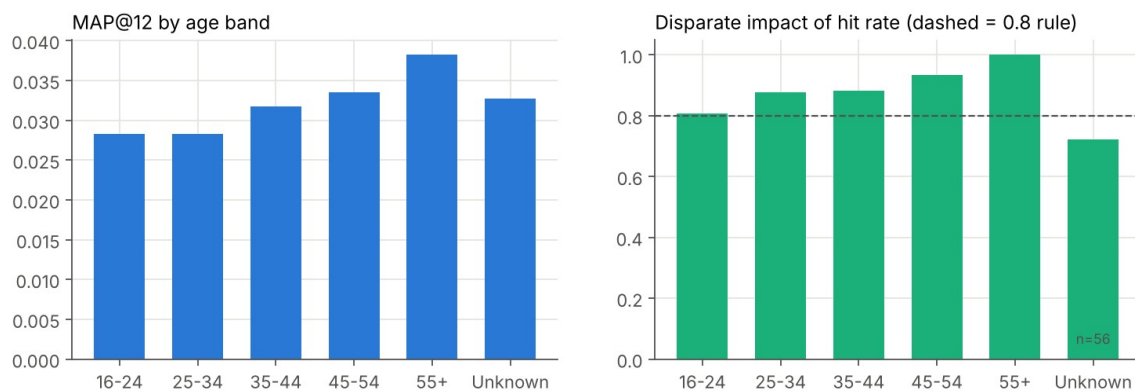


Figure 16. MAP@12 and disparate impact of hit rate by age band. Unknown has only 56 customers.

1. Quality goes up with age: MAP@12 is 0.0283 for 16 to 24 and 0.0283 for 25 to 34, vs 0.0382 for 55+
2. The youngest band passes the four-fifths rule, but only just (0.81)
3. The Unknown band (56 customers, 14 purchases among candidates) is reported but too small to judge, so it is not flagged
4. Why: young customers buy more new things (Step 3: 2.0% repeat items vs 2.8% for 55+), and the model's strongest signals are about repeat and related purchases. So the gap comes from shopping behavior, but the effect is still that younger customers get worse recommendations

Demographic parity and equalized odds. Here a positive prediction means the item makes the customer's top 12.

Table 14. Selection rate, true positive rate and false positive rate by age band

Age band	Selection rate	True positive rate	False positive rate	Bought candidates
16-24	15.3%	0.450	0.151	1,193
25-34	14.4%	0.434	0.143	1,598
35-44	14.1%	0.429	0.139	555
45-54	14.9%	0.475	0.147	918
55+	14.7%	0.518	0.145	597

Age band	Selection rate	True positive rate	False positive rate	Bought candidates
Unknown	15.1%	0.429	0.151	14

- Demographic parity: every band gets 14.1% to 15.3% of its candidates into the top 12. Difference 0.012, ratio 0.92
- Equalized odds: the false positive rate gap is small (0.012), but the true positive rate goes from 0.429 (35 to 44) to 0.518 (55+), a gap of 0.089. So the model is equally selective for everyone, but better at catching what older customers will buy

Item side: popularity bias. I call an item head if it is in the top 20% of sellers in the history window, and long tail otherwise.

Table 15. Exposure of head vs long-tail items in the test week

Measure	LightGBM ranker	Best-seller list
Share of slots going to head items	95.3%	100%
Share of slots going to long-tail items	4.7%	0%
Share of actual purchases that are long tail	34.1%	34.1%
Distinct items recommended	7,045	12
Coverage of the candidate catalog	29.0%	0.05%
Exposure Gini (1 = all slots on one item)	0.90	

This is the bigger fairness issue for a marketplace. Long-tail items are a third of what people actually buy, but get less than 5% of the slots. Popular items get more popular, which hurts new and smaller brands.

5.4 Mitigations tested

1. Reweighting (pre-processing): training rows weighted so each age band counts equally. MAP@12 0.0304, lowest disparate impact 0.81. It did not help: the young bands are already the biggest groups, so the weights barely change anything
2. Removing age features (fairness through unawareness): dropped customer age, age missing flag, age gap, buyers' average age and age-band rank. MAP@12 0.0307 vs 0.0309 (a 0.8% drop). The lowest disparate impact improves from 0.81 to 0.86 and the gap between the best and worst band shrinks slightly. Older customers lose a little, younger ones gain a little
3. Long-tail re-ranking (post-processing): new score = score percentile minus lambda times popularity percentile. At lambda 0.1 the long-tail share rises from 4.7% to 5.5% and coverage from 29% to 32%, but MAP@12 drops 9%. At 0.5 it drops 35%
4. Group thresholds (proposed, not needed): selection rates are already equal across bands, so per-band thresholds would not fix the true positive gap
5. Augmentation (proposed): a longer history window for young and new customers, and trend-based generators, to give the model more to go on where it is weakest

Table 16. Accuracy vs long-tail exposure for different re-ranking strengths

Lambda	MAP@12	Hit rate	Long-tail share of slots	Coverage	Exposure Gini
0.0	0.0309	13.1%	4.7%	29.0%	0.900
0.1	0.0280	13.0%	5.5%	31.8%	0.889
0.2	0.0247	12.6%	6.2%	34.2%	0.877

Lambda	MAP@12	Hit rate	Long-tail share of slots	Coverage	Exposure Gini
0.3	0.0226	12.3%	6.9%	36.3%	0.865
0.5	0.0200	11.5%	8.4%	40.9%	0.843
0.8	0.0173	10.2%	10.9%	47.6%	0.813

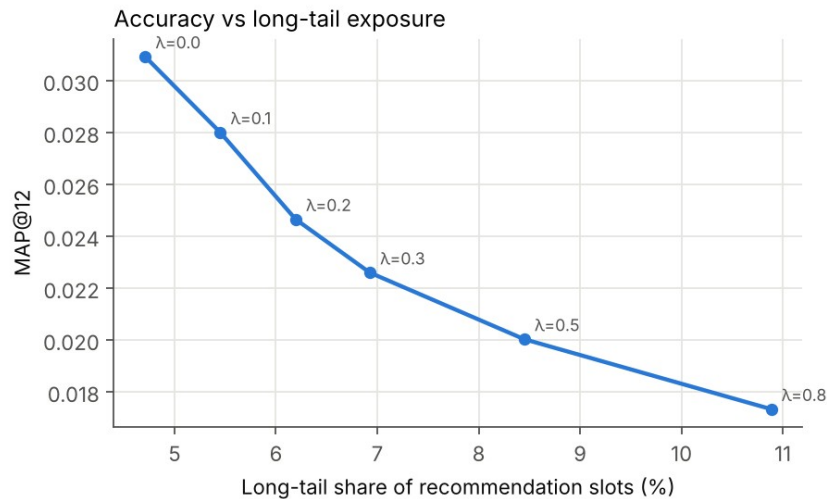


Figure 17. Each step of lambda buys more long-tail exposure at a clear cost in MAP@12.

5.5 What I would do

1. Ship the version without age features. It costs almost nothing in accuracy, narrows the age gap and avoids using age directly. Keep age only for monitoring
2. Do not re-rank the main row. Instead give long-tail items one or two dedicated "new for you" slots, so the cost is contained and can be measured in an A/B test
3. Monitor hit rate by age band every week with 0.8 as the alert line, plus the long-tail share of slots
4. Privacy: the data is pseudonymous, no customer-level data is in the repo, and the GenAI reason line is blocked from mentioning age, gender or any personal trait. In production, customers should be able to switch personalization off

Step 6: Final Presentation and Communication

Two deliverables for mixed audiences: Technical presentation (Jupyter slides / LaTeX Beamer) for peers. Business-facing presentation (PowerPoint / Canva) for executives (ROI, risks, strategy). 8 to 12 slides per deck recommended. Deliverables: Two slide decks (technical + business).

1. Technical deck (LaTeX Beamer, reports/technical_deck.pdf, 12 slides): problem and metric, data and split, EDA, pipeline, features and selection, model comparison, tuning and overfitting, SHAP and PDP, fairness, mitigations, deployment and GenAI, limitations and next steps
2. Business deck (PowerPoint, reports/business_deck.pptx, 10 slides): the problem in plain words, what we built, results in business terms, what drives a pick, a pilot ROI scenario with its assumptions, risks and fairness, how we would use it, rollout plan and next steps

Step 7: GitHub Profile and Upload

Create a public GitHub repo structured like an Open Source project: Include: src/ for scripts, notebooks/, data/, models/ directories. Deliverables: GitHub repo link + final report + reproducible code.

Repo: https://github.com/mix-afk/Matthew_Martinez_Capstone

- src/recsys/: the package (data, candidates, features, ranker, evaluate, fairness, explain, genai, serving, api)
- notebooks/: 01 data, 02 EDA and features, 03 models, 04 explainability and fairness, 05 GenAI and serving
- data/: README with download and sampling steps. The H&M files are not committed, because the competition rules do not allow sharing them
- models/: the three rankers, feature list, tuned settings and test metrics
- reports/: this report, both decks, all figures and result tables
- docs/: data dictionary, deployment guide, GenAI notes, model card
- tests/ and .github/workflows/ci.yml: 19 unit tests and lint, run on every push
- README, LICENSE (MIT), requirements.txt, pyproject.toml, Makefile, Dockerfile

The commit history follows the build: scaffold, data, features, models, explainability and fairness, app and GenAI, CI, docs, report.

Step 8: Deployment and MLOps (optional)

Local deployment (required): deploy the best model via Flask, FastAPI, or Dash.

Optional cloud. MLOps practices: reproducible environments (requirements.txt, Docker), config-driven runs & experiment tracking (MLflow, W&B, etc.), CI checks (lint, unit tests), basic monitoring plan, versioning & rollback plan. Provide a demo (GIF/screencast).

The best model runs as a nightly batch plus a FastAPI service. The batch scores every active customer and writes their top 12, product details and "bought together" neighbors. The API loads those files and answers in milliseconds, so no model runs inside a request. New customers get best sellers, or best sellers for their age band if the caller passes one.

- Endpoints: / (demo page), /health, /recommendations/{customer_id}, /similar/{article_id}, /customers/demo, /docs
- Environment: pinned requirements.txt, a slim requirements-api.txt and a Dockerfile with a health check
- Config-driven runs: configs/config.yaml; MLflow tracking in mlruns/
- CI: GitHub Actions runs ruff and pytest on every push
- Monitoring plan: health and latency, data freshness, feature drift (PSI on top SHAP features), weekly offline MAP@12 and hit rate by age band, GenAI fallback rate
- Versioning and rollback: git tags for code, MLflow runs for models, dated artifact folders; rollback means pointing the API at the previous folder and restarting

The full guide is docs/DEPLOYMENT.md, and demo/demo.gif shows the app.

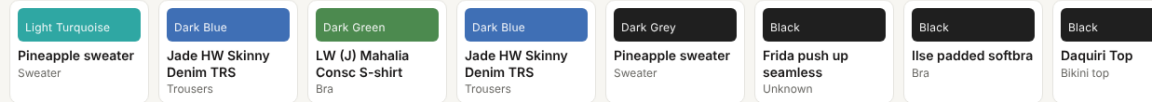
You may also like

Two-stage recommender demo on H&M data. Recommendations use data up to Sep 15, 2020.

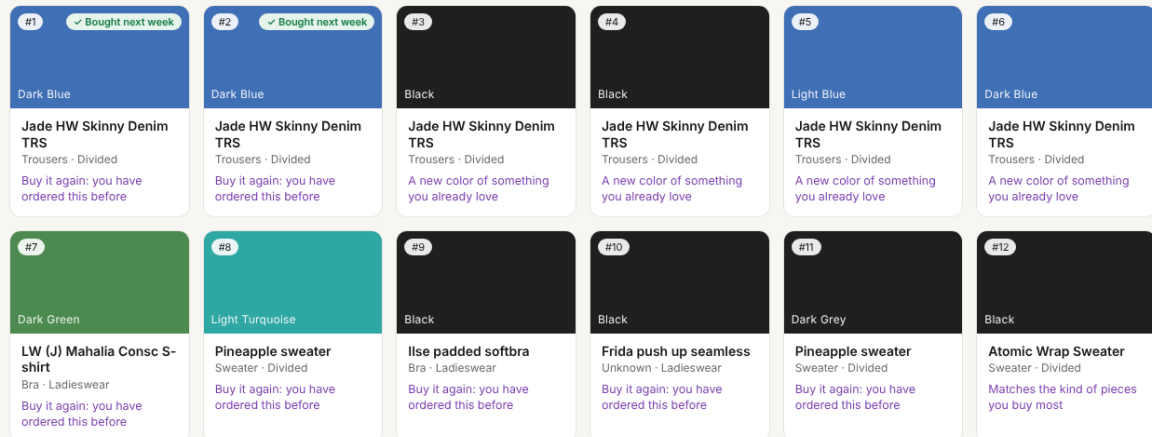
Demo customer 1 or paste a customer_id Recommend ☐ AI reasons

Customer ff57364873... Past items 8 Bought next week 2 of 12

Recently bought last 8 weeks



Recommended for you candidates + LightGBM ranker



Green badge = the customer really bought that item in the following week (16 to 22 Sep 2020), which the model never saw.

Figure 18. The demo page: a customer's recent purchases, their top 12 with a reason line, and a check mark on items they really bought the following week.

Step 9: Use of Generative AI (optional)

Use LLMs to auto-generate EDA summaries or data dictionaries. Build GenAI-enhanced applications (e.g., LLM-backed recommenders, chatbots). Deliverable: document how Generative AI was used in the project; include code + examples in the GitHub repo or presentation; demo video.

1. In the product: each recommendation has a one-line reason. Code first turns the ranker's own features into facts (bought before, another color of something you bought, often bought together, trending). Claude then writes a short line from those facts only. A guardrail blocks any mention of age, gender or other personal traits, and if the line fails the check or the API is down, a template is used instead. The LLM never decides what to recommend, only how to explain it, so it cannot invent a reason that is not true
 2. For documentation: a helper profiles each table (type, missing %, unique values, range) and asks Claude for a first draft of the data dictionary. I checked every line against the data and recomputed the counts and ranges on the full files. The EDA summaries were drafted the same way from notebook outputs and then checked against the numbers
 3. While building: I used Claude as a coding assistant for scaffolding, first drafts of modules and tests, and debugging. All results come from the notebooks in the repo
- Code: src/recsys/genai.py. Examples: notebook 05. Notes: docs/GENAI.md.

Conclusion and next steps

A two-stage recommender beats a best-seller list by a wide margin on a week it never saw (MAP@12 0.0309 vs 0.0094), stays explainable with SHAP, and has known, measurable fairness gaps with a cheap fix for the age gap. The biggest limit is the candidate pool: 89% of next week's purchases never reach the ranker.

1. Better candidates: a longer history window, "trending in your categories" and bought-together pairs
2. Run on the full data instead of 20% of customers
3. Ship the version without age features, with long-tail slots, and A/B test it against best sellers on the KPIs in Step 1
4. For Dynamix: the same pipeline can run per brand on marketplace order data once it is connected to our order system

References

- H&M Personalized Fashion Recommendations, Kaggle competition (2022): kaggle.com/competitions/h-and-m-personalized-fashion-recommendations
- Parquet copy of the competition data, Derrick Mwiti: kaggle.com/datasets/derrickmwiti/hm-personalized-fashion-recommendations
- Wp-Zhang, H-M-Fashion-RecSys (45th of 3,006, private score 0.02996): github.com/Wp-Zhang/H-M-Fashion-RecSys
- Ke et al. (2017), LightGBM: A Highly Efficient Gradient Boosting Decision Tree, NeurIPS
- Lundberg and Lee (2017), A Unified Approach to Interpreting Model Predictions (SHAP), NeurIPS
- Hu, Koren and Volinsky (2008), Collaborative Filtering for Implicit Feedback Datasets (ALS), ICDM
- Feldman et al. (2015), Certifying and Removing Disparate Impact, KDD; Hardt, Price and Srebro (2016), Equality of Opportunity in Supervised Learning, NeurIPS